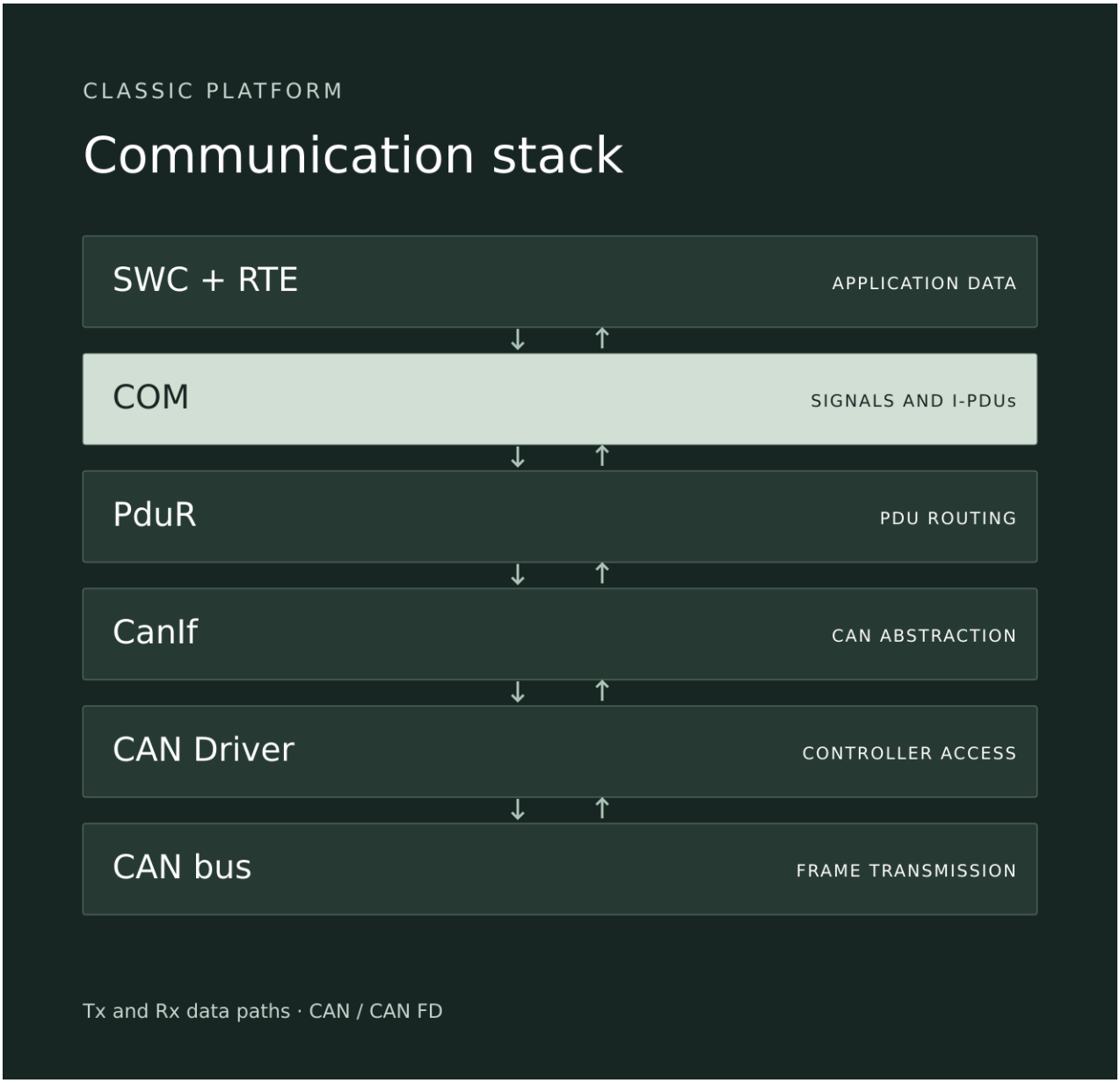


AUTOSAR COM

Ôn phỏng vấn senior
8 năm kinh nghiệm

AUTOSAR Classic
Tháng 9 năm 2026

Mục lục

1 Nắm rõ phạm vi COM module và Communication Stack

2 Phân biệt Signal Signal Group I PDU và CAN frame

3 Mô tả toàn bộ luồng Tx

4 Mô tả toàn bộ luồng Rx

5 Các lớp buffer và tính nhất quán dữ liệu

6 Packing endianness và kiểu dữ liệu

7 Signal Group và shadow buffer

8 Transfer Property và Tx Mode

9 TMC TMS và Tx filter

10 Rx filter và tương tác với giá trị notification và timeout

11 Update Bit và sự khác nhau giữa cập nhật và thay đổi

12 Reception Deadline Monitoring

13 Invalid handling và timeout

14 Tx timing và chu kỳ offset repetition MDT

Mục lục tiếp theo

15 Transmission Deadline Monitoring

16 Immediate và Deferred processing

17 I PDU Group và điều khiển trạng thái

18 Signal gateway và PDU gateway

19 TP dynamic signal và CAN FD

20 Bus off recovery và trách nhiệm của CanSM

21 Network Management sleep và wake up

22 E2E checksum alive counter và SecOC

23 RTE semantics và concurrency

24 Timing performance và đường trễ

25 Phương pháp debug có hệ thống

26 Các test case senior nên tự thiết kế được

27 Mười câu hỏi tình huống cần trả lời trôi chảy

28 Cách trình bày để thể hiện đúng mức senior

Tài liệu tham khảo

Tổng quan

Với **senior 8 năm kinh nghiệm**, phỏng vấn COM stack thường đánh giá ba năng lực: **hiểu chính xác cơ chế, phân tích được tương tác giữa các module và tìm được nguyên nhân lỗi trên hệ thống thực tế**.

Tài liệu tổng hợp theo **AUTOSAR Classic, trọng tâm CAN/CAN FD**. Các tên API và một số chi tiết hành vi cần đối chiếu với AUTOSAR release và BSW vendor của dự án.

Kiến thức nền tảng

1 Nắm rõ phạm vi COM module và Communication Stack

Module	Trách nhiệm cần nhớ
RTE	Kết nối SWC với dịch vụ truyền thông; quản lý truy cập dữ liệu theo cấu hình RTE
COM	Pack/unpack signal, signal group, Tx mode, filtering, update bit, deadline monitoring, signal gateway
PduR	Route PDU giữa các module theo routing path
CanIf	Trừu tượng hóa CAN Driver; xử lý mapping Tx/Rx PDU, CAN ID, controller, PDU mode
Can Driver	Điều khiển CAN controller, mailbox, interrupt/polling
CanTrcv	Điều khiển CAN transceiver, chế độ hoạt động và wake-up liên quan
CanTp	Segmentation/reassembly cho truyền thông theo transport protocol
ComM	Điều phối nhu cầu communication của user/channel
CanSM	Quản lý trạng thái truyền thông CAN, controller/transceiver và bus-off recovery
CanNm / Nm	Network management, phối hợp duy trì mạng thức và đi ngủ
BswM	Thực thi rule/action theo trạng thái hệ thống, như điều khiển I-PDU group
EcuM	Các phần quản lý vòng đời ECU và wake-up
IpduM	Multiplexing/demultiplexing và container I-PDU theo cấu hình
E2E / SecOC	Bảo vệ dữ liệu theo mục tiêu safety hoặc security tương ứng

Cần trả lời rõ:

- COM không trực tiếp xử lý CAN arbitration.
- COM không thực hiện bus-off recovery.
- PduR không mặc định giải mã và xử lý giá trị signal.
- ComM FULL_COMMUNICATION không đồng nghĩa tất cả COM I-PDU đã được start.
- UDS qua CAN thường đi theo Dcm ↔ PduR ↔ CanTp ↔ CanIf ↔ Can Driver, không phải qua COM.

2 Phân biệt Signal Signal Group I PDU và CAN frame

Khái niệm	Ý nghĩa
Signal	Một dữ liệu được COM xử lý, như tốc độ hoặc trạng thái công tắc
Group Signal	Một thành viên của signal group
Signal Group	Tập signal cần được trao đổi nhất quán như một nhóm
I-PDU	Đơn vị dữ liệu COM trao đổi với lớp dưới
CAN frame	Đơn vị truyền trên CAN, gồm identifier, control fields, payload…
PDU ID	Handle dùng trong phần mềm; không phải CAN ID

PDU ID có phạm vi theo module. Cùng một luồng dữ liệu có thể mang các handle khác nhau tại COM, PduR và CanIf.

Với giao tiếp CAN thông thường, một COM I-PDU có thể ánh xạ vào payload một frame. Không được khái quát rằng mọi I-PDU luôn tương ứng một CAN frame, vì còn TP, multiplexing và container.

3 Mô tả toàn bộ luồng Tx

Luồng điển hình:

SWC → RTE → COM → PduR → CanIf → Can Driver → CAN bus

Khi ứng dụng ghi dữ liệu:

1. RTE chuyển dữ liệu xuống COM tại thời điểm phù hợp với kiểu truy cập.
2. COM cập nhật signal hoặc dữ liệu nhóm.
3. COM xử lý update bit, điều kiện chọn Tx mode và trigger.
4. Khi đủ điều kiện truyền, COM yêu cầu PduR gửi I-PDU.
5. CanIf/Driver xử lý yêu cầu xuống controller.
6. Khi truyền hoàn tất, confirmation quay ngược lên các lớp đã cấu hình.

Bốn mốc không được nhầm:

Mốc	Chứng minh được điều gì?
Rte_Write hoàn tất	Lời gọi RTE đã được xử lý theo contract
Com_SendSignal trả thành công	Yêu cầu ghi signal được COM xử lý; chưa chứng minh frame đã ra bus
CanIf_Transmit trả E_OK	Yêu cầu được chấp nhận theo cơ chế lớp dưới
Tx confirmation	Việc truyền đã được xác nhận theo semantics của stack

Tx confirmation không chứng minh ứng dụng ở ECU đích đã sử dụng dữ liệu. Trên CAN, ACK cũng không định danh ECU ứng dụng cần nhận.

4 Mô tả toàn bộ luồng Rx

Luồng điển hình:

CAN controller → Can Driver → CanIf → PduR → COM → RTE → SWC

Khi debug, phải tìm được các bước:

- Controller nhận frame.
- Driver báo Rx.
- CanIf match CAN ID, HRH và cấu hình Rx PDU.
- PduR tìm đúng routing path.
- COM kiểm tra và xử lý I-PDU theo cấu hình.
- Signal được chấp nhận, cập nhật và thông báo.
- RTE cung cấp dữ liệu cho SWC.

Nhìn thấy frame trên CANoe chưa chứng minh ECU đã nhận frame đó vào COM.

Tương tự, Com_RxIndication được gọi chưa đủ để khẳng định mọi signal bên trong đều được cập nhật: còn update bit, invalid handling, filtering và cách xử lý immediate/deferred.

5 Các lớp buffer và tính nhất quán dữ liệu

Khi interviewer hỏi “buffer có cập nhật không?”, cần xác định họ đang nói đến:

- CAN mailbox.
- Buffer của CanIf/PduR, nếu có.

- COM I-PDU buffer.
- COM signal storage.
- Signal group shadow buffer.
- RTE buffer.
- Biến mà runnable đang sử dụng.

Cách bố trí bộ nhớ thực tế tùy vendor; không nhất thiết mỗi khái niệm là một vùng RAM riêng.

Ví dụ: frame mới đã được copy vào buffer phục vụ deferred processing, nhưng signal bị Rx filter loại bỏ. Khi đó **raw payload có thể đã thay đổi, còn giá trị signal cung cấp cho ứng dụng vẫn giữ giá trị trước đó**.

Đây là lý do không nên trả lời chung chung "buffer cập nhật" hoặc "buffer không cập nhật".

6 Packing endianness và kiểu dữ liệu

Cần thành thạo:

- Bit position, bit size.
- Little endian, big endian, opaque.
- Signal nằm vắt qua nhiều byte.
- Signed/unsigned và sign extension.
- Kiểu dữ liệu ứng dụng khác độ rộng signal trên bus.
- Initial value, invalid value và các bit không sử dụng.
- Mảng cố định và dữ liệu độ dài động.

Ví dụ: signal signed 12 bit có raw value 0xFFF biểu diễn -1. Khi đưa vào kiểu signed rộng hơn, phải sign-extend đúng.

Những lỗi thực tế hay gặp:

- Dùng sai quy ước start bit khi chuyển DBC sang cấu hình AUTOSAR.
- Nhầm cách biểu diễn Motorola bit numbering.
- Truyền con trỏ sai kiểu vào Com_SendSignal.
- Sai signedness khiến giá trị âm trở thành số dương lớn.
- Đọc physical value trên CANoe nhưng so với raw value trong COM.

COM không mặc định thực hiện mọi phép đổi đơn vị vật lý trong DBC. Phải biết phép đổi physical = raw × factor + offset đang được thực hiện ở tool hiển thị, RTE, transformer hay ứng dụng.

7 Signal Group và shadow buffer

Mục tiêu của signal group là giữ **tính nhất quán giữa các thành viên**.

Ví dụ {PositionX, PositionY, Timestamp} cần được sử dụng như một bộ dữ liệu.

Với API shadow buffer truyền thống:

- Tx: Com_UpdateShadowSignal() cập nhật từng thành viên.
- Com_SendSignalGroup() chuyển dữ liệu nhóm sang vùng dữ liệu truyền của COM và xử lý yêu cầu gửi.
- Rx: Com_ReceiveSignalGroup() lấy một snapshot vào shadow buffer.
- Com_ReceiveShadowSignal() đọc từng thành viên từ snapshot đó.

Cần nhớ:

- Cập nhật shadow signal chưa có nghĩa frame được gửi.
- Signal group không đồng nghĩa I-PDU group.
- Tính nhất quán của nhóm không tự giải quyết mọi race condition giữa nhiều task cùng ghi.
- Nếu dùng signal group array API, phải hiểu đường xử lý riêng; không áp nguyên xi mô hình shadow API.

Cơ chế truyền nhận và giám sát

8 Transfer Property và Tx Mode

Transfer Property mô tả cách một lần cập nhật signal/group tạo trigger.

Transfer Property	Ý nghĩa chính
PENDING	Cập nhật dữ liệu, không tự tạo event trigger
TRIGGERED	Mỗi yêu cầu ghi tạo trigger
TRIGGERED_ON_CHANGE	Tạo trigger khi dữ liệu khác giá trị so sánh đang lưu
TRIGGERED_WITHOUT_REPETITION	Tạo trigger không yêu cầu chuỗi repetition
TRIGGERED_ON_CHANGE_WITHOUT_REPETITION	Chỉ trigger khi thay đổi, không yêu cầu repetition

Tx Mode quy định cách I-PDU được truyền:

Tx Mode	Hành vi
PERIODIC	Truyền theo chu kỳ
DIRECT	Truyền theo event trigger
MIXED	Kết hợp periodic và event
NONE	Không có lịch periodic/direct tự động từ mode này

Hai câu bẫy:

- **PENDING có được truyền không?** Có. Dữ liệu được mang theo khi I-PDU truyền vì chu kỳ hoặc trigger từ signal khác.
- **TRIGGERED có luôn làm frame ra bus ngay không?** Không. Phải xét Tx mode hiện tại, I-PDU state, MDT, scheduling và lớp dưới.

“On change” cũng không đồng nghĩa “khác giá trị lần cuối được ECU đích nhận thành công”.

9 TMC TMS và Tx filter

- **TMC — Transmission Mode Condition:** kết quả điều kiện cho signal/group signal có cấu hình liên quan.
- **TMS — Transmission Mode Selector:** quyết định chọn ComTxModeTrue hay ComTxModeFalse cho I-PDU.

- Trong cơ chế thông thường, các TMC được kết hợp theo OR.
- Không có điều kiện filter tham gia thì quy tắc mặc định cần được hiểu đúng; không tự suy ra TMS là FALSE.

Điểm phải thuộc: Rx filter loại bỏ signal; Tx filter được dùng để chọn transmission mode, không loại bỏ việc ghi signal ở phía phát. Vì vậy, Tx filter FALSE không có nghĩa dữ liệu mới bị bỏ hoặc I-PDU chắc chắn ngừng truyền. [1]

Ví dụ cấu hình:

TMS	Mode
TRUE	MIXED, chu kỳ nhanh
FALSE	PERIODIC, chu kỳ chậm

Filter FALSE vẫn có thể làm I-PDU tiếp tục truyền theo chu kỳ chậm.

Senior còn cần giải thích mode switch ảnh hưởng thế nào đến cyclic timer, repetition và MDT trong release đang dùng.

10 Rx filter và tương tác với giá trị notification và timeout

Các thuật toán thường gặp gồm ALWAYS, NEVER, MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_DIFFERS_MASKED_OLD, NEW_IS_WITHIN, NEW_IS_OUTSIDE, ONE_EVERY_N.

Ví dụ, với filter kiểm tra khoảng:

- Signal hiện tại: 40.
- Dữ liệu mới nhận: 150.
- Khoảng chấp nhận: 0…100.
- Không có timeout replacement hoặc hành động khác xen vào.

Kết quả: signal mới bị loại; ứng dụng tiếp tục đọc 40, normal reception notification cho signal đó không được tạo vì lần nhận bị lọc.

Filter FALSE không phải một sự kiện "tăng số lần timeout". Timer deadline monitoring vận hành theo thời gian và điều kiện reception của nó, độc lập với việc giá trị có được ứng dụng chấp nhận hay không.

Với **I-PDU-based Rx deadline monitoring**, việc giám sát không xét giá trị các signal. Do đó I-PDU vẫn đến hợp lệ có thể giữ communication alive dù giá trị signal bị filter loại. [2]

Khi trả lời chi tiết, phải nói rõ signal có update bit không; không áp quy tắc I-PDU-based cho mọi trường hợp.

11 Update Bit và sự khác nhau giữa cập nhật và thay đổi

Update bit thể hiện signal/group đã được phía phát cập nhật theo cơ chế COM.

Update bit bằng 1 không chứng minh giá trị khác lần trước.

Ví dụ ứng dụng ghi Speed = 50 liên tiếp: dữ liệu vẫn có thể được đánh dấu updated dù giá trị không đổi.

Ở phía nhận:

- Có update bit và bit bằng 0: signal/group không được xử lý như một update mới.
- Bit bằng 1: đủ điều kiện đi tiếp vào xử lý signal.
- Signal có update bit có thể được giám sát deadline riêng.

Ở phía phát, cần biết cấu hình clear update bit theo sự kiện như transmit, confirmation hoặc trigger-transmit.

Tình huống quan trọng: frame tới mỗi 10 ms nhưng update bit của một signal luôn bằng 0. Signal đó vẫn có thể timeout dù bus traffic bình thường.

Update bit không thay thế alive counter, E2E counter hay timestamp.

12 Reception Deadline Monitoring

Cần giải thích được bốn câu hỏi: **khi nào bắt đầu, khi nào reload, khi nào hết hạn và hết hạn thì làm gì?**

Tham số	Ý nghĩa cần nhớ
ComFirstTimeout	Giám sát giai đoạn trước reception đầu tiên sau khi bắt đầu giám sát
ComTimeout	Timeout dùng cho giám sát reception tiếp theo
ComRxDataTimeoutAction	Hành động lên dữ liệu khi timeout
ComTimeoutNotification	Callback timeout nếu cấu hình

ComTimeout = 0 hoặc không cấu hình thì không giám sát signal/group đó; không thể chỉ đặt ComFirstTimeout để thay thế. [3]

Các điểm khác phải nhớ:

- ComFirstTimeout = 0 thường có nghĩa chưa giám sát trước lần nhận đầu tiên, không phải timeout ngay.
- Không update bit: dùng cơ chế I-PDU-based.
- Có update bit: có thể giám sát riêng từng signal/group.
- Với các signal không update bit trong cùng I-PDU, cần hiểu quy tắc chọn timeout chung từ cấu hình.
- Disable reception DM không đồng nghĩa stop reception.

Timeout action	Giá trị sau timeout
NONE	Giữ giá trị hiện tại
REPLACE	Thay bằng initial value
SUBSTITUTE	Thay bằng timeout substitution value

Đừng nhầm timeout action với chính sách an toàn của ứng dụng. Giữ giá trị cũ vẫn có thể đi kèm trạng thái dữ liệu không hợp lệ.

Câu bẫy: "Giá trị vẫn hợp lý thì có thể dùng tiếp không?"

Không thể quyết định chỉ từ numeric value; phải xét age, timeout status và yêu cầu chức năng.

13 Invalid handling và timeout

Trường hợp	Ý nghĩa
Timeout	Không có reception/update đủ điều kiện trong khoảng cho phép
Invalid value	Có dữ liệu tới nhưng chứa giá trị invalid đã quy ước
Filter rejection	Có dữ liệu tới nhưng điều kiện filter không chấp nhận
E2E error	Dữ liệu không đạt kiểm tra E2E
Bus-off	Controller gặp trạng thái lỗi CAN

Với invalid handling:

- NOTIFY: gọi invalid notification; không tiếp tục xử lý bình thường như một signal hợp lệ.
- REPLACE: thay bằng initial value, rồi xử lý tiếp theo quy tắc tương ứng.

Invalid reception vẫn có thể reload reception deadline timer. Vì thế "không timeout" không chứng minh dữ liệu có giá trị sử dụng. [4]

Khi phân tích dữ liệu, luôn tách **value, validity và freshness**.

14 Tx timing và chu kỳ offset repetition MDT

Tham số	Cần hiểu
ComTxModeTimePeriod	Chu kỳ periodic
ComTxModeTimeOffset	Offset cho lịch truyền
ComTxModeNumberOfRepetitions	Số lần lặp bổ sung
ComTxModeRepetitionPeriod	Khoảng thời gian repetition
ComMinimumDelayTime	Giới hạn khoảng cách truyền theo quy tắc MDT

Nếu number of repetitions là 2, chuỗi tương ứng **lần đầu + 2 lần lặp**. Nhưng không được chỉ đếm ba lần gọi API: cơ chế N-times có liên quan đến các confirmation thành công và tương tác với request mới. [5]

Cần phân biệt:

- **Repetition:** lặp ở COM theo cấu hình.
- **Retry:** thử lại khi request xuống lớp dưới thất bại.
- **CAN retransmission:** cơ chế của CAN controller/protocol.

Ví dụ tính lịch lý tưởng: repetition period 10 ms, hai repetitions, không MDT cản trở, không lỗi và không jitter → có thể quan sát các lần truyền gần 0, 10, 20 ms.

Trong thực tế còn scheduling, confirmation, arbitration và mode switch. MDT là cơ chế của COM; nó không điều khiển trực tiếp thời điểm controller thắng arbitration trên bus.

15 Transmission Deadline Monitoring

Tx DM giám sát việc truyền/xác nhận ở phía dưới; nó không chứng minh application đã cập nhật đúng hạn.

Các câu cần trả lời:

- Timer bắt đầu tại đâu trong trường hợp đang xét?
- Request thất bại ngay thì timer thế nào?
- Request mới có restart timer đang chạy không?
- Một confirmation có kết thúc toàn bộ giám sát không?
- N-times và mode NONE có quy tắc riêng gì?
- Khi timeout có cancellation support không?

Không học thuộc một câu "Tx confirmation tới là dừng timer" cho mọi cấu hình. Với N-times phải xét số confirmation cần thiết và các quy tắc đặc biệt.

Khi gặp Tx timeout, kiểm tra:

- Request có tới PduR/CanIf không?
- Có bị callout chặn không?
- CanIf/controller đang ở mode nào?
- Mailbox/queue có bị nghẽn không?
- Driver interrupt/polling có hoạt động không?
- Confirmation có bị mất hoặc sai mapping không?
- Timeout có đủ cho repetition và worst-case bus delay không?

16 Immediate và Deferred processing

Kiểu	Đặc điểm	Rủi ro cần đánh giá
IMMEDIATE	Xử lý trong ngữ cảnh indication tương ứng	Tăng thời gian xử lý trong ngữ cảnh caller
DEFERRED	Hoãn phần xử lý sang COM main function	Tăng latency và phụ thuộc scheduling

Không được mặc định immediate luôn chạy trong ISR: caller có thể xuất phát từ polling task.

Với deferred, phải biết phần nào được thực hiện ngay khi indication đến và phần nào bị hoãn. Không nên nói "mọi thứ đợi đến Com_MainFunctionRx".

Một case phỏng vấn hay:

> *Frame đến mỗi 1 ms, COM Rx main function chạy mỗi 10 ms. Ứng dụng có thấy đủ 10 mẫu không?*

Không được mặc định có. Signal communication thường cung cấp giá trị hiện tại; khả năng giữ đủ từng mẫu phụ thuộc buffering/queueing cụ thể. Nếu requirement yêu cầu không mất event, kiến trúc truyền thông phải hỗ trợ điều đó.

Tích hợp và trạng thái hệ thống

17 I PDU Group và điều khiển trạng thái

I-PDU group dùng để điều khiển tập I-PDU theo trạng thái vận hành.

Cần nắm:

- Start/stop group.
- Khác biệt giữa start có khởi tạo và không khởi tạo.
- Enable/disable reception DM.
- I-PDU thuộc nhiều group.
- Restart ảnh hưởng thế nào đến dữ liệu, filter state và timer.
- PDU đang pending ở lớp dưới khi stop sẽ được xử lý thế nào.

Không coi stop COM group là cách chắc chắn thu hồi frame đã bàn giao cho hardware.

Những trạng thái phải kiểm tra riêng:

Cấp	Ví dụ
Communication demand	ComM user request
Network state	CanSM state
Controller	STARTED / STOPPED / SLEEP
CanIf PDU mode	ONLINE / OFFLINE / TX_OFFLINE…
Routing	PduR routing path group
COM	I-PDU group started/stopped
Monitoring	Rx DM enabled/disabled

Nhiều lỗi “COM không gửi” thực chất là một trong các điều kiện này chưa được thỏa mãn.

18 Signal gateway và PDU gateway

Tiêu chí	COM signal gateway	PduR PDU gateway
Đơn vị route	Signal/group	PDU

Tiêu chí	COM signal gateway	PduR PDU gateway
Thay đổi cách bố trí signal	Có thể theo mapping	Không tự unpack/repack từng signal
Đổi chu kỳ đích	Có thể qua cấu hình Tx COM	Không phải chức năng rate conversion của PDU routing
Chi phí xử lý	Có xử lý dữ liệu ở mức signal	Thường ít xử lý payload hơn

AUTOSAR phân biệt PDU routing với rate conversion qua COM signal gateway. [6]

Senior cần phân tích được:

- Ghép signal từ nhiều nguồn có tạo bộ dữ liệu khác tuổi không?
- Nguồn timeout nhưng đích vẫn gửi periodic thì freshness được thể hiện thế nào?
- Gateway có làm mất ý nghĩa update bit không?
- E2E protection được giữ xuyên suốt hay kết thúc và tạo lại tại gateway?
- Worst-case gateway latency là bao nhiêu?

Gateway tiếp tục gửi không chứng minh dữ liệu nguồn tiếp tục được cập nhật.

19 TP dynamic signal và CAN FD

Cần biết khi nào đi qua communication interface và khi nào qua transport protocol.

Với COM large I-PDU qua TP, cần nhận diện các API như:

- Com_StartOfReception
- Com_CopyRxData
- Com_TpRxIndication
- Com_CopyTxData
- Com_TpTxConfirmation

Điểm cần hiểu:

- Buffer ownership trong quá trình truyền/nhận TP.
- Khi nào dữ liệu nhận hoàn chỉnh được công bố.
- Xử lý thất bại giữa chừng.
- Busy status khi tài nguyên đang phục vụ TP.
- Actual length và maximum length của dynamic signal.

CAN FD cho phép payload lớn hơn Classical CAN, nhưng **CAN FD không đồng nghĩa ISO-TP**, và không phải mọi COM I-PDU trên CAN FD đều cần CanTp.

20 Bus off recovery và trách nhiệm của CanSM

Luồng thông báo điển hình:

CAN controller/Driver → CanIf_ControllerBusOff → CanSM_ControllerBusOff

CanSM điều phối recovery, các chuyển trạng thái liên quan và báo cho module khác theo cấu hình.

Cần giải thích:

- Vì sao controller bus-off.
- Recovery phần cứng khác policy recovery của BSW thế nào.
- Restart controller khác cho phép Tx trở lại thế nào.
- Fast recovery và slow recovery.
- Tiêu chí xem recovery đã thành công.
- Tương tác với ComM, BswM, DEM và COM group/DM.

Các tham số thường gặp:

- CanSMBorTimeL1
- CanSMBorTimeL2
- CanSMBorCounterL1ToL2
- CanSMBorTimeTxEnsured
- CanSMBorTxConfirmationPolling

L1/L2 là các mức thời gian recovery; không phải các mức lỗi của COM. [7]

Không có một thời gian recovery cố định đúng cho mọi ECU.

Sau recovery phải kiểm tra dữ liệu nào được gửi đầu tiên: dữ liệu cũ, initial value hay dữ liệu application vừa cập nhật. Đây là câu hỏi về tích hợp toàn stack.

21 Network Management sleep và wake up

Cần mô tả được chuỗi điều kiện từ communication request đến frame thực sự được gửi, và chiều ngược lại khi đi ngủ.

Phải phân biệt:

- Application request.
- ComM state.

- CanSM state.
- CanNm state.
- Controller/transceiver state.
- BswM action.
- COM I-PDU group state.

Các tình huống nên luyện:

- ECU không ngủ vì user vẫn giữ request.
- ECU thức lại ngay vì wake-up source còn active.
- Network đã thức nhưng COM chưa start group.
- Startup phát sinh timeout giả vì Rx DM được bật trước khi ECU nguồn sẵn sàng.
- Sleep bị trì hoãn bởi diagnostic communication hoặc điều kiện network management.

Với selective wake-up/partial networking, cần hiểu thêm transceiver capability, PNC mapping và NM configuration.

22 E2E checksum alive counter và SecOC

Cơ chế	Mục đích chính
CAN CRC	Phát hiện lỗi ở mức frame CAN
COM Rx DM	Phát hiện thiếu reception/update trong thời gian yêu cầu
Update bit	Đánh dấu cập nhật signal/group
Alive/sequence counter	Theo dõi tiến triển hoặc thứ tự dữ liệu
E2E	Phát hiện các lỗi truyền thông theo profile và fault model
SecOC	Xác thực và bảo vệ freshness theo mô hình security

Cần trả lời:

- Vì sao có CAN CRC vẫn cần E2E?
- Vì sao Rx DM không phát hiện được mọi trường hợp stale data?
- Vì sao counter còn tăng mà một signal vẫn có thể bị frozen?
- Vì sao CRC thông thường không chống giả mạo?
- Ai quyết định application được sử dụng dữ liệu khi E2E báo lỗi?

Không mặc định "COM tự tính checksum/counter". Phải chỉ ra lớp thực hiện trong dự án: transformer, E2E integration, callout, SWC hoặc cơ chế riêng.

23 RTE semantics và concurrency

Cần phân biệt:

- Rte_Read và Rte_IRead.
- Rte_Write và Rte_IWrite.
- Explicit access và implicit access.
- Unqueued data và queued communication.
- Data received event và thời điểm runnable thực sự được schedule.

Ví dụ: COM đã có giá trị mới nhưng runnable dùng implicit read vẫn đang xử lý snapshot của lần thực thi hiện tại. Đây có thể là hành vi đúng.

Về concurrency, cần hiểu:

- Rx interrupt/polling có thể chạy đồng thời với COM main function.
- Application có thể ghi khi Tx processing đang đọc.
- Nhiều task cùng dùng signal group shadow buffer.
- Exclusive area/SchM.
- Reentrancy theo API và theo đối tượng.
- Multicore, partition và chi phí chuyển dữ liệu.

volatile không thay thế mutual exclusion hoặc atomicity.

24 Timing performance và đường trễ

Một mô hình phân tích hữu ích:

$$L_{\text{end-to-end}} = L_{\text{source task}} + L_{\text{RTE/COM Tx}} + L_{\text{queue/arbitration}} + L_{\text{bus}} + L_{\text{Rx processing}} + L_{\text{destination task}}$$

Mỗi thành phần có thể chứa thời gian chờ và thời gian thực thi; tránh cộng trùng.

Cần tính đến:

- Task period, phase và jitter.
- COM main function period.
- MDT, repetition và cyclic scheduling.
- CAN priority, bus load, blocking và arbitration.
- Deferred processing.
- Gateway delay.

- Task phía nhận.

Không được kết luận signal latency là 10 ms chỉ vì PDU period là 10 ms.

Khi tối ưu, phải giải thích trade-off:

- Immediate giảm một phần latency nhưng tăng tải trong callback context.
- Main function nhanh hơn tăng CPU load.
- Gom nhiều signal tiết kiệm overhead nhưng có thể làm phức tạp freshness và priority.
- Bus load trung bình thấp chưa chứng minh worst-case response time đạt yêu cầu.

Debug kiểm thử và phỏng vấn

25 Phương pháp debug có hệ thống

Với lỗi **"ứng dụng ghi nhưng không thấy frame"**, kiểm tra theo đường dữ liệu:

1. Runnable có chạy, giá trị có được ghi đúng không?
2. RTE explicit/implicit đã chuyển dữ liệu xuống COM chưa?
3. COM signal/PDU ID và buffer có đúng không?
4. I-PDU group đã start chưa?
5. Transfer property, TMS, mode, MDT và Tx main function thế nào?
6. COM có gọi PduR không, return value là gì?
7. Route, CanIf PDU mode và controller có cho phép truyền không?
8. Mailbox, CAN state và confirmation ra sao?

Với lỗi **"frame có nhưng ứng dụng không đổi"**:

1. CAN ID, channel, DLC và payload có đúng không?
2. Driver/CanIf có nhận đúng Rx PDU không?
3. PduR có route đến đúng COM I-PDU không?
4. COM group/callout có chặn không?
5. Update bit, invalid value và filter thế nào?
6. Deferred main function có chạy không?
7. COM signal value khác RTE/runnable snapshot ở thời điểm nào?

Cách làm tốt: ghi timestamp và dữ liệu tại các ranh giới module để xác định đoạn đầu tiên lệch kỳ vọng. Breakpoint có thể làm thay đổi timing, nhất là với timeout và bus-off.

26 Các test case senior nên tự thiết kế được

Nhóm	Test quan trọng
Packing	Boundary bit, cross-byte, endian, signed min/max
Tx	Từng transfer property kết hợp từng mode
TMS	TRUE/FALSE, nhiều TMC, chuyển mode khi đang repetition
Filter	Pass/fail, boundary, mask, trạng thái sau timeout
Update bit	0/1, ghi lại cùng giá trị, chính sách clear
Rx DM	First timeout, normal timeout, disable/enable, restart
Invalid	NOTIFY/REPLACE và tương tác filter
Group	Snapshot nhất quán, thiếu thao tác commit, concurrent access
Timing	Jitter, CPU overload, bus overload, deferred latency
Lớp dưới	Request thất bại, mất confirmation, mailbox bận
Lifecycle	Startup, sleep, wake-up, stop/start group
Bus-off	Lỗi đơn lẻ, lỗi lặp, chuyển L1/L2, dữ liệu sau phục hồi
Gateway	Nguồn mất, đích vẫn gửi, dữ liệu khác tuổi
TP	Thiếu buffer, abort, sai length, cập nhật khi đang busy

Một test tốt kiểm tra đồng thời **bus trace, signal value, trạng thái chất lượng dữ liệu và notification**. Chỉ kiểm tra giá trị cuối cùng sẽ bỏ sót nhiều lỗi.

27 Mười câu hỏi tình huống cần trả lời trôi chảy

Câu hỏi	Ý chính phải trả lời
Filter FALSE có gây timeout không?	Tách Rx/Tx; Rx filtering không phải timeout event; xét DM và update bit
Frame vẫn đến nhưng signal timeout?	Update bit, signal-specific DM, routing/processing và timing
Tx filter FALSE thì có gửi không?	Có thể; filter chọn mode, mode FALSE vẫn có thể truyền
Com_SendSignal = E_OK nhưng không có frame?	API success khác bus transmission; kiểm tra trigger/mode/state/lớp dưới

Câu hỏi	Ý chính phải trả lời
Signal PENDING có ra bus không?	Có, khi I-PDU được gửi vì nguyên nhân khác
CANoe thấy mới, application thấy cũ?	Kiểm tra từng lớp buffer, filter, deferred processing và RTE snapshot
Tx confirmation có đảm bảo ECU đích xử lý?	Không; cần application-level acknowledgment nếu requirement đòi hỏi
Tăng timeout để hết lỗi được không?	Chỉ sau khi xác định timing budget và yêu cầu phát hiện lỗi
Bus-off hết nhưng COM vẫn không gửi?	Kiểm tra toàn bộ state/mode/group và pending request
Gateway đang gửi đều, nguồn đã chết thì sao?	Dữ liệu đích có thể cũ; phải có chính sách freshness/validity rõ ràng

28 Cách trình bày để thể hiện đúng mức senior

Với mỗi tình huống, trả lời theo năm bước:

1. **Chốt giả thiết:** Tx/Rx, release, vendor, mode, update bit, immediate/deferred.
2. **Giải thích cơ chế:** state, buffer, timer và trigger nào tham gia.
3. **Nêu điều kiện ngoại lệ:** TP, repetition, mode switch, group restart…
4. **Đưa cách kiểm chứng:** API boundary, timestamp, trace và configuration.
5. **Đánh giá tác động:** latency, stale data, bus load, CPU và hành vi ứng dụng.

Ví dụ trả lời câu “Rx filter FALSE thì thế nào?”:

> *“Tôi xác nhận đây là Rx signal filter. Giá trị mới bị loại, nên giá trị signal cung cấp cho ứng dụng giữ nguyên nếu không có hành động khác xen vào. Raw I-PDU buffer có thể đã được cập nhật. Normal signal notification không xảy ra cho lần nhận bị lọc. Deadline monitoring cần xét riêng: I-PDU-based DM không phụ thuộc giá trị signal; nếu signal có update bit thì phải xét điều kiện cập nhật của signal đó.”*

Cuối cùng, nên chuẩn bị **ba case thực tế từ dự án của chính bạn**: một lỗi Tx/timing, một lỗi Rx/freshness và một lỗi state management hoặc bus-off. Với mỗi case, nói rõ triệu chứng, bằng chứng, root cause, cách sửa và test chống tái phát. Đây là phần giúp người phỏng vấn phân biệt người thuộc cấu hình với người thực sự làm chủ COM stack.

Tài liệu tham khảo

Các tài liệu AUTOSAR dưới đây tương ứng với các trích dẫn trong nội dung. Khi áp dụng cho dự án, đối chiếu đúng release và tài liệu của BSW vendor.

[1] AUTOSAR Specification of Communication R21 11

Filtering và Transmission Mode Selection
Mở tài liệu AUTOSAR

[2] AUTOSAR Specification of Communication R23 11

Reception Deadline Monitoring và SWS_Com_00738
Mở tài liệu AUTOSAR

[3] AUTOSAR Specification of Communication R4 3 1

ComTimeout và SWS_Com_00333
Mở tài liệu AUTOSAR

[4] AUTOSAR Specification of Communication R25 11

Invalid handling và Reception Deadline Monitoring
Mở tài liệu AUTOSAR

[5] AUTOSAR Specification of Communication R24 11

Replication of Signal Transmission Requests và SWS_Com_00305
Mở tài liệu AUTOSAR

[6] AUTOSAR Specification of PDU Router R20 11

PDU gateway và rate conversion
Mở tài liệu AUTOSAR

[7] AUTOSAR Specification of CAN State Manager R20 11

Bus off recovery và các tham số CanSMBor
Mở tài liệu AUTOSAR